

Question Index - All 40 Questions

Q#	Question
Q1	AI in DevOps - use cases, AIOps, where AI adds value
Q2	GitHub Copilot - AI pair programmer, how it works, IaC generation
Q3	Amazon CodeWhisperer / Q Developer - AWS-native AI coding
Q4	AI for code review - automated PR review, Codium, CodeRabbit
Q5	ChatOps basics - Slack/Teams bots, incident management, deployment triggers
Q6	Jira basics - issues, sprints, boards, workflows, DevOps integration
Q7	Jira and CI/CD - linking commits, build status, deployment tracking
Q8	Confluence - documentation, runbooks, postmortems, knowledge base
Q9	Backstage basics - developer portal, software catalog, templates
Q10	Slack/Teams integration - webhooks, bots, notifications, incident channels
Q11	AI for incident management - PagerDuty AIOps, root cause suggestions
Q12	AI for monitoring - anomaly detection, predictive alerting, noise reduction
Q13	AI for testing - test generation, visual regression, flaky test detection
Q14	AI for security - AI-powered SAST, vulnerability prioritization
Q15	GitHub Copilot for IaC - Terraform, Kubernetes YAML, Ansible generation
Q16	AI-powered log analysis - pattern detection, root cause from logs
Q17	Backstage plugins - CI/CD, monitoring, cost, custom plugins
Q18	Backstage software templates - golden paths, scaffolding new services
Q19	Backstage TechDocs - docs-as-code, MkDocs integration
Q20	Jira automation - rules, triggers, smart commits, workflow automation
Q21	Jira and Kubernetes - deployment tracking, environment visibility
Q22	ChatOps advanced - Slack workflows, deployment approvals, incident bots

Q23	PagerDuty - on-call schedules, escalation policies, incident workflows
Q24	OpsGenie - alert routing, schedules, integration with monitoring
Q25	Statuspage - public status pages, incident communication
Q26	Runbook automation - documented procedures to automated remediation
Q27	Postmortem culture - blameless postmortems, templates, follow-ups
Q28	Knowledge management - wikis, decision records, architectural docs
Q29	AI for cost optimization - Spot.io, Kubecost, CloudHealth AI recommendations
Q30	AI for capacity planning - predictive scaling, demand forecasting
Q31	Building an internal developer platform - Backstage, self-service, golden paths
Q32	DevEx (Developer Experience) - measuring and improving developer productivity
Q33	AI governance in DevOps - responsible AI, prompt security, model drift
Q34	Collaboration at scale - cross-team practices, inner source, platform teams
Q35	GitHub Copilot generated insecure code - SQL injection in IaC
Q36	ChatOps bot deployed to production without approval
Q37	AI monitoring alert fatigue - too many false positive anomalies
Q38	Backstage adoption failed - developers not using the portal
Q39	Postmortem action items never completed - breaking the cycle
Q40	Design a complete developer experience platform for 200 engineers

SECTION 1 - Junior Core Concepts (Q1–Q10)

Q1 AI in DevOps - use cases, AIOps

The Simple Answer

AI in DevOps (AIOps) applies machine learning to automate and improve operations: anomaly detection in metrics (find unusual patterns before they become outages), log analysis (cluster millions of log lines into meaningful patterns), incident management (correlate alerts, suggest root causes, reduce noise), predictive scaling (forecast traffic and scale infrastructure before demand hits), and code assistance (generate IaC, review PRs, suggest fixes).

The Analogy - An Experienced SRE Who Never Sleeps

A senior SRE who has seen a thousand incidents can glance at metrics and say "that pattern looks like a memory leak - check the Java heap." AI does the same thing but across all your services simultaneously, 24/7. It learns normal patterns, detects anomalies faster than dashboards, and suggests root causes based on historical correlations. It does not replace the SRE - it gives them superpowers.

Where AI Adds Real Value Today

High value (proven): Anomaly detection in metrics and logs. Alert noise reduction (correlating 100 alerts into 1 incident). Predictive autoscaling. Code completion for IaC. Automated security scanning prioritization. Medium value (emerging): Automated root cause analysis. Test generation. ChatOps-driven remediation. Cost optimization recommendations. Low value (overhyped): Fully autonomous deployment decisions. "Self-healing" without human oversight. Replacing DevOps engineers entirely.

****The interview perspective:**** Show awareness of AI capabilities without overpromising. "AI assists DevOps by reducing noise, detecting anomalies faster, and automating repetitive tasks. It augments engineers rather than replacing them. The human still makes the critical decisions - AI provides better data to decide with."

Q2 GitHub Copilot - AI pair programmer

The Simple Answer

GitHub Copilot is an AI coding assistant that suggests code completions, generates entire functions, and writes boilerplate based on context and comments. Powered by large language models trained on public code. Available in: VS Code, JetBrains, Neovim, and GitHub.com.

DevOps Use Cases

Terraform: Type a comment "# Create an S3 bucket with versioning and encryption" → Copilot generates the complete Terraform resource block. Kubernetes YAML: Start typing a Deployment manifest → Copilot completes with sensible defaults (liveness probes, resource limits, security context). Ansible playbooks: Write a task name

→ Copilot generates the task with correct module syntax. Bash scripts: Describe the function in a comment → Copilot writes the implementation. Dockerfiles: Type "FROM python" → Copilot generates a multi-stage build with best practices.

Copilot Chat

Ask questions in natural language: "How do I configure Terraform remote state with S3 and DynamoDB locking?" → Copilot explains and generates the code. "What's wrong with this Kubernetes manifest?" → Copilot identifies issues (missing resource limits, running as root, no readiness probe).

****The critical caveat:**** Copilot generates code based on patterns, not understanding. It can produce: insecure code (hardcoded secrets, SQL injection), outdated patterns (deprecated APIs), incorrect logic (subtle bugs), and license-violating code. ALWAYS review AI-generated code. Never blindly commit Copilot suggestions. See scenario Q35.

Q3 Amazon CodeWhisperer / Q Developer

The Simple Answer

Amazon Q Developer (formerly CodeWhisperer) is AWS's AI coding assistant. Specialized for AWS services - generates Boto3 code, CloudFormation templates, CDK constructs, and AWS CLI commands with higher accuracy than general-purpose tools. Available in: VS Code, JetBrains, AWS Cloud9, and the AWS Console.

Advantages Over Copilot for AWS Work

Trained on AWS documentation and APIs. Better at: Boto3 pagination patterns, CloudFormation intrinsic functions, IAM policy documents, and AWS-specific best practices. Built-in security scanning - flags insecure code patterns as you type. Free tier available for individual developers.

AWS Console Integration

Amazon Q in the AWS Console answers questions about your infrastructure: "How many EC2 instances are running in us-east-1?" "What's the cost of this RDS instance per month?" "Help me write an IAM policy for Lambda to read from S3."

Q4 AI for code review - automated PR review

The Simple Answer

AI-powered code review tools analyze pull requests and provide automated feedback: bug detection, security issues, performance suggestions, style improvements, and documentation gaps. They complement human reviewers - catching mechanical issues so humans focus on architecture and logic.

Tools

CodeRabbit - AI PR reviewer. Adds comments on PRs with: bug risk assessment, security suggestions, performance concerns, and style improvements. Learns team preferences over time. Amazon CodeGuru Reviewer - AWS service. Detects: resource leaks, concurrency issues, input validation gaps, and AWS API best

practices. SonarQube + AI - SonarQube's latest versions incorporate AI for more accurate issue detection and fewer false positives.

The Balance

AI code review is a first pass - catching obvious issues before human reviewers spend time. It does NOT replace human review for: architecture decisions, business logic correctness, code readability judgment, or mentoring through feedback. The best workflow: AI reviews first (automated, instant) → human reviews second (focused on what AI cannot judge).

Q5 ChatOps basics - Slack/Teams bots

The Simple Answer

ChatOps brings operations into chat platforms. Instead of SSHing into servers or logging into dashboards, you interact with infrastructure through Slack or Teams commands. Deployments, incident management, and monitoring happen in chat - visible to the entire team.

The Analogy - A Shared Control Room

Traditional operations is like each engineer working alone in a dark room. ChatOps is a shared control room where everyone sees the same screens, hears the same alerts, and can act from the same interface. When someone deploys, everyone sees it. When an alert fires, everyone knows. When someone fixes an issue, the resolution is documented in chat history.

Common ChatOps Patterns

/deploy api v2.1.0 to staging - Trigger deployment from Slack. The bot calls the CI/CD pipeline, streams status updates to the channel. /status production - Check production health. The bot queries monitoring and reports. /incident create "Payment API 500 errors" - Create an incident channel, page on-call, start the incident timeline. Alert notifications - Prometheus/PagerDuty sends alerts to a Slack channel. The team discusses and resolves in the channel - the entire incident is documented.

Q6 Jira basics - issues, sprints, DevOps integration

The Simple Answer

Jira is the most widely used project management tool in software development. It tracks work through: issues (stories, bugs, tasks, epics), boards (Kanban or Scrum), sprints (time-boxed iterations), and workflows (status transitions: To Do → In Progress → In Review → Done).

DevOps Relevance

Jira is the bridge between business requirements and engineering execution. A Jira ticket defines WHAT to build. The developer creates a branch (feature/JIRA-123-user-auth), writes code, and commits with the ticket

reference. CI/CD links builds and deployments back to the ticket. The product owner sees: requirement → code → build → deployment - complete traceability.

Key Integrations

Git integration - Branch names and commit messages containing JIRA-123 automatically link to the ticket. CI/CD - Build results (pass/fail) appear on the ticket. Deployment status shows which environment has this change. Slack - Jira notifications in Slack channels. Transition tickets from Slack. Confluence - Link Jira epics to Confluence design documents.

Q7 Jira and CI/CD - linking, deployment tracking

The Simple Answer

Smart Commits - Commit messages with Jira syntax trigger actions: "JIRA-123 #done #time 2h Fixed login timeout." This transitions the ticket to Done and logs 2 hours of work - all from a commit message.

Build Integration - Jenkins, GitHub Actions, GitLab CI report build status to Jira. The ticket shows: "Build #142: SUCCESS" with a link to the build log. Deployment Tracking - Jira Deployments show: which version is in which environment. "JIRA-123 is deployed to staging (v2.1.0) but not yet in production." Product owners see deployment status without asking engineers. Release Hub - Group tickets into releases. Track release progress. Generate release notes from completed tickets.

Q8 Confluence - documentation, runbooks

The Simple Answer

Confluence is Atlassian's wiki platform for team documentation. In DevOps, Confluence stores: architecture diagrams, runbooks (step-by-step incident response procedures), postmortem reports, onboarding guides, and decision records.

Key DevOps Documents

Runbooks - Step-by-step procedures for common incidents: "Database connection pool exhausted," "Disk full on web servers," "Certificate expiring." Each runbook includes: symptoms, diagnosis steps, resolution steps, and escalation path. The goal: a junior engineer can follow the runbook at 3 AM and resolve the incident without escalation.

Postmortems - After every significant incident, document: what happened, timeline, root cause, impact, what went well, what went poorly, and action items with owners and due dates. See Q27 for postmortem culture.

Architecture Decision Records (ADRs) - Document why decisions were made: "We chose PostgreSQL over DynamoDB because..." Future engineers understand the context behind architectural choices.

Q9 Backstage basics - developer portal

The Simple Answer

Backstage (created by Spotify, donated to CNCF) is an open-source developer portal that centralizes: software catalog (every service, library, and tool in your organization), documentation (TechDocs), CI/CD status, monitoring dashboards, API documentation, and infrastructure costs - all in one place.

The Analogy - A Company Directory for Software

In a large organization, finding answers is hard: "Who owns the payment service? Where are its docs? What pipeline deploys it? What's its uptime?" Backstage is the company directory for software - one portal that answers all these questions for every service. Instead of searching Slack, GitHub, Confluence, and Grafana separately, developers go to Backstage.

Key Features

Software Catalog - Every service registered with: owner (team), lifecycle (production, deprecated), documentation link, API definition, dependencies, and CI/CD pipeline. Search and discover any component in the organization.

Software Templates - Golden path templates for creating new services. "Create a new Python microservice" generates: repository, CI/CD pipeline, Kubernetes manifests, monitoring dashboards, and Backstage catalog entry - all from a single template. Ensures every new service follows organizational standards.

TechDocs - Documentation as code. Markdown files in the repository, rendered in Backstage. Every service's documentation is discoverable in one place.

Plugins - Extensible with 100+ community plugins: Kubernetes (cluster/Pod status), PagerDuty (on-call schedules), GitHub Actions (pipeline status), Cost (cloud spend per service), SonarQube (code quality), and custom plugins for internal tools.

Q10 Slack/Teams integration - webhooks, bots

The Simple Answer

Incoming webhooks - Send messages TO Slack from external tools. `curl -X POST -d '{"text":"Deploy v2.1.0 complete"}' https://hooks.slack.com/services/XXX`. Used by: CI/CD (build notifications), monitoring (alert notifications), deployment tools (status updates).

Slack bots - Interactive applications that respond to commands and messages. `/deploy`, `/status`, `/incident` commands. Bots can: trigger pipelines, query monitoring, create incident channels, page on-call engineers.

Workflow Builder - No-code automation in Slack. Trigger: message in a channel containing "incident." Action: create an incident channel, invite the on-call team, post the runbook link.

Best Practices for Notifications

Avoid noise - Only send actionable notifications. "Build failed" is actionable. "Build passed" is noise (unless specifically requested). Use threads - post initial notification, then update the thread with progress (not the main channel). Channel strategy - `#alerts-critical` (pages on-call), `#alerts-warning` (reviewed daily), `#deployments` (all deploy notifications), `#incidents` (active incident channels).

SECTION 2 - Mid-Level Production Patterns (Q11–Q28)

Q11 AI for incident management - PagerDuty AIOps

The Simple Answer

During an incident, 100+ alerts fire from different monitoring tools. Humans cannot process this volume fast enough. AI correlates alerts into a single incident, suggests the probable root cause based on historical incidents, and recommends runbooks or actions.

PagerDuty AIOps - Reduces alert noise by 90%+ through intelligent grouping. 100 alerts from a network partition become 1 incident: "Network connectivity loss in us-east-1a affecting 15 services." Suggests: "Similar incidents in the past were caused by: 1) NLB health check misconfiguration (60%), 2) Security group change (25%), 3) Route table modification (15%)."

BigPanda - Correlates alerts across monitoring tools (Prometheus, Datadog, CloudWatch, custom). Topology-aware - understands service dependencies. Identifies the root cause service, not just the symptoms.

Q12 AI for monitoring - anomaly detection

The Simple Answer

Traditional monitoring uses static thresholds: alert if CPU > 80%. But normal CPU varies by time of day, day of week, and season. 80% at noon on Black Friday is normal. 80% at 3 AM on a Tuesday is not.

ML-based anomaly detection learns normal patterns and alerts on deviations. Datadog - Anomaly detection on any metric. Learns seasonal patterns (daily, weekly). Alerts when behavior deviates from expected. Dynatrace Davis AI - Automatically discovers topology, learns baselines, and identifies root causes without manual configuration. Amazon CloudWatch Anomaly Detection - ML on CloudWatch metrics. Creates bands of expected behavior. Alerts when metrics leave the band.

Predictive Alerting

Instead of alerting when disk is 90% full (reactive), predict when it WILL be full based on the growth rate (proactive). "At current rate, disk will be full in 6 hours." This gives the team time to act before impact.

Q13 AI for testing - test generation

The Simple Answer

AI generates unit tests from existing code, detects visual regressions in UI screenshots, and identifies flaky tests through pattern analysis.

Diffblue Cover - Generates JUnit tests for Java code automatically. Analyzes methods and creates test cases covering different execution paths. Codium AI - Generates test cases for Python, JavaScript, and TypeScript. Suggests edge cases the developer might miss. AppliTools - Visual AI for UI testing. Compares screenshots pixel-by-pixel with AI that ignores minor rendering differences (anti-aliasing, font smoothing) and flags real visual regressions. Reduces false positives from traditional pixel comparison by 99%.

AI for Flaky Test Detection

Analyze test execution history. ML identifies: tests whose pass rate drops before a flaky failure (early warning), tests correlated with specific environments or times (environment-dependent), and tests that fail in clusters (shared root cause).

Q14 AI for security - AI-powered SAST

The Simple Answer

Traditional SAST tools match patterns - they find SQL injection by looking for string concatenation in queries. AI-powered SAST understands code semantics - it follows data flow across functions and files to detect vulnerabilities that pattern matching misses.

Snyk Code - AI-powered SAST. Faster and more accurate than traditional scanners. Fewer false positives. Understands data flow across the entire codebase. GitHub Advanced Security (CodeQL) - Queries code as data. Powerful but requires query writing. Semgrep - Pattern-based (not AI) but fast and DevOps-friendly. Good complement to AI tools.

AI for Vulnerability Prioritization

Not all CVEs are equally dangerous. AI prioritizes based on: is the vulnerable code reachable from the internet? Is the vulnerability actively exploited in the wild? How severe is the impact in YOUR environment? This reduces the vulnerability backlog from 500 items to 20 critical items that actually need immediate attention.

Q15 GitHub Copilot for IaC

The Simple Answer

Copilot excels at IaC because Terraform, Kubernetes YAML, and Ansible playbooks are highly structured and repetitive - exactly what AI models are best at predicting.

Effective Patterns

Comment-driven generation: "# Create a VPC with public and private subnets across 3 AZs" → Copilot generates the complete Terraform VPC module. Pattern continuation: Write one security group rule → Copilot suggests the remaining rules following the same pattern. Boilerplate elimination: Type the resource type → Copilot fills in all required and recommended properties with sensible defaults.

Limitations for IaC

Copilot does not know YOUR infrastructure - it generates generic patterns. It may suggest: outdated API versions, insecure defaults (0.0.0.0/0 ingress), non-compliant configurations (missing encryption), or resources that do not fit your architecture. Always review against: your organization's Terraform modules, security policies, and naming conventions.

Q16 AI-powered log analysis

The Simple Answer

Processing millions of log lines manually is impossible. AI clusters similar log patterns, detects anomalies (new error patterns that never appeared before), and correlates log events with metrics to suggest root causes.

Elastic AI/ML - Anomaly detection on log data. Detects: unusual log volume spikes, new error message patterns, and categorizes logs automatically. Datadog Log Patterns - Automatically groups similar log lines. Highlights: new patterns (appearing for the first time - potential new bugs) and trending patterns (increasing frequency - degrading service).

Q17 Backstage plugins - extending the portal

The Simple Answer

Backstage's power is in plugins. Each plugin adds a card or page to the service catalog.

Kubernetes plugin - Shows Pod status, deployments, and events for each service directly in Backstage. No kubectl needed. GitHub Actions plugin - Shows CI/CD pipeline status and history. PagerDuty plugin - Shows on-call schedule and active incidents for each service. Cost plugin - Shows cloud costs per service (Kubecost integration). SonarQube plugin - Shows code quality metrics.

Custom Plugins

Build plugins for internal tools: your deployment platform, your feature flag system, your internal API documentation. The plugin framework uses React (frontend) and Express (backend). A simple plugin takes 2-3 days to build.

Q18 Backstage software templates - golden paths

The Simple Answer

A golden path is an opinionated, paved road for common development tasks. Instead of each developer configuring a new service differently, the template provides a standardized setup.

What a Template Generates

"Create a new Python microservice" → GitHub repository with standard structure → Dockerfile with security best practices → Helm chart for Kubernetes deployment → GitHub Actions CI/CD pipeline → Prometheus monitoring (ServiceMonitor) → Grafana dashboard → Backstage catalog entry → PagerDuty service and escalation policy. In 5 minutes, a developer has a production-ready service skeleton following all organizational standards.

Why Templates Matter

Without templates: each developer builds differently, documentation is inconsistent, CI/CD is configured ad-hoc, and monitoring gaps are common. With templates: every service starts from the same foundation. Compliance is built in. Onboarding is faster. The platform team maintains the templates - updates propagate to new services automatically.

Q19 Backstage TechDocs

The Simple Answer

TechDocs renders Markdown documentation from source code repositories in the Backstage portal. Developers write docs as Markdown alongside their code. Backstage builds and serves them. Every service's documentation is discoverable in one place - no searching Confluence, Google Docs, and README files separately.

How It Works

docs/ directory in the repository with MkDocs configuration (mkdocs.yml). CI builds the docs on every push. Backstage indexes and serves them. Search across ALL services' documentation from one search bar.

Q20 Jira automation - rules, triggers

The Simple Answer

Jira Automation creates rules: when TRIGGER happens, if CONDITION is met, then DO ACTION.

DevOps Automation Rules

Auto-transition on PR merge: When a PR referencing JIRA-123 is merged → transition the ticket to "Done." Auto-assign on sprint start: When sprint starts → assign unassigned stories to team members based on workload. Alert on stale tickets: When a ticket has been "In Progress" for more than 5 days → send Slack notification to the assignee and team lead. Auto-create sub-tasks: When a "Story" is created → create sub-tasks: "Write tests," "Update docs," "Review."

Q21 Jira and Kubernetes - deployment tracking

The Simple Answer

Jira Deployments (via the Deployments feature) show which Jira tickets are deployed to which environment. CI/CD pipelines send deployment data to Jira's REST API after each deploy. The Jira ticket shows: "Deployed to staging on May 15, Production on May 16." Product owners see real-time deployment status without asking engineers. Release managers know exactly which changes are in each environment.

Q22 ChatOps advanced - deployment approvals, incident bots

The Simple Answer

Deployment approvals in Slack: CI pipeline reaches the production deploy stage → sends a Slack message with Approve/Reject buttons → authorized user clicks Approve → pipeline continues → deployment starts → status updates posted to the thread. The entire approval is documented in Slack history.

Incident bots: When PagerDuty pages, the bot: creates a dedicated #incident-2025-05-16 channel, invites the on-call team, posts the alert details and runbook link, starts a timeline, and after resolution, generates a postmortem template. Tools: PagerDuty Slack integration, Opsgenie Slack, custom bots (Hubot, Bolt).

Q23 PagerDuty - on-call, escalation

The Simple Answer

PagerDuty is the standard for incident management and on-call scheduling.

On-call schedules - Define who is on-call, when, with rotation rules. Primary and secondary on-call. Handoff times. Holiday overrides. Escalation policies - If the primary on-call does not acknowledge in 5 minutes → page the secondary. If no response in 10 minutes → page the team lead. If no response in 15 minutes → page the engineering manager. Incident workflows - When triggered: page on-call → create Slack channel → add stakeholders → start status page update. After resolution: generate timeline → create postmortem document → schedule review meeting. Alert integrations - Prometheus, Grafana, Datadog, CloudWatch, custom webhooks. Alert deduplication, suppression during maintenance windows, and intelligent grouping.

Q24 OpsGenie - alert routing

The Simple Answer

OpsGenie (Atlassian) provides alert management with deep Jira and Confluence integration. Alert routing rules: if alert source is "database" → route to the DBA team. if severity is "P1" → page primary AND secondary on-call. if time is outside business hours → use the on-call schedule. if time is business hours → post to Slack channel only.

Q25 Statuspage - incident communication

The Simple Answer

Statuspage (Atlassian) provides public and internal status pages. During an incident, update the status page: "Investigating → Identified → Monitoring → Resolved." Customers subscribe to updates (email, SMS, webhook). Internal status pages show service health to employees. Components represent individual services: API (Operational), Dashboard (Degraded Performance), Payments (Major Outage).

Why Status Pages Matter

Without a status page, during an outage: support tickets flood in, customers panic on Twitter, and executives ask "what's happening?" every 5 minutes. With a status page: customers see the current status, subscribe to updates, and trust that you are handling it. Support ticket volume drops 80% during incidents with status page updates.

Q26 Runbook automation

The Simple Answer

Runbooks evolve through stages: Level 1 - Wiki page: Step-by-step instructions a human follows manually. "SSH to the server. Run this command. Check this metric." Level 2 - Semi-automated: A script performs the steps, human initiates and monitors: "Run ./remediate-disk-full.sh on web-1." Level 3 - Fully automated: Event-Driven Ansible, PagerDuty Automation Actions, or custom bots detect the condition and execute the runbook without human intervention. The human is notified after the fact.

The Progression

Most organizations start at Level 1. Mature organizations have Level 2 for common incidents and Level 3 for well-understood, low-risk remediations (restart a crashed service, clear disk cache, rotate a certificate). High-risk actions (database failover, network changes) stay at Level 1-2 with human oversight.

Q27 Postmortem culture - blameless

The Simple Answer

A postmortem (also called incident review or retrospective) is a structured analysis of an incident. The goal is to learn and prevent recurrence - not to blame individuals.

Blameless Culture

The principle: "The person who caused the outage is the person who needs to feel MOST safe speaking up." If people fear punishment, they hide mistakes. Hidden mistakes repeat. Blame creates silence. Blamelessness creates learning. Focus on: what went wrong in the SYSTEM (process, tooling, design) - not who did something wrong.

Postmortem Template

Summary: What happened in 2-3 sentences. Impact: Duration, affected users, revenue impact. Timeline: Minute-by-minute events from detection to resolution. Root cause: The underlying system weakness, not "Bob made a typo." What went well: Alerts fired quickly, runbook worked, team responded fast. What went poorly: Detection was slow, runbook was outdated, communication was unclear. Action items: Specific, assigned, with due dates. "Add input validation to the deploy script (Alice, by May 30)."

The Critical Step

Action items must be tracked and completed. A postmortem without follow-through is a document that collects dust. Track action items in Jira. Review completion in the next team meeting. See scenario Q39.

Q28 Knowledge management

The Simple Answer

Architecture Decision Records (ADRs) - Document WHY decisions were made. "ADR-042: We chose PostgreSQL over DynamoDB because our data model requires complex joins." Future engineers understand the context. Stored in Git alongside the code or in Confluence.

Runbooks - Step-by-step incident response procedures. See Q26.

Onboarding guides - Day-by-day plan for new team members. "Day 1: Set up laptop, clone repos. Day 2: Deploy to staging. Day 3: On-call shadow."

Decision: Information close to the code goes in Git (ADRs, API docs, README). Information for broader consumption goes in Confluence (architecture overviews, onboarding, postmortems).

SECTION 3 - Senior Architecture (Q29–Q34)

Q29 AI for cost optimization

The Simple Answer

Cloud costs grow 20-30% annually without optimization. AI identifies waste and recommends actions.

Spot.io (NetApp) - ML-based Spot Instance management. Predicts interruptions and migrates workloads proactively. 60-80% savings on compute. Kubecost - Kubernetes cost visibility. Cost per namespace, per deployment, per label. Recommendations: right-size requests/limits, identify idle resources. CloudHealth / Apptio - Multi-cloud cost management. AI recommendations: "This RDS instance has been at 5% CPU for 30 days - downsize from db.r5.xlarge to db.r5.large, saving \$200/month."

Q30 AI for capacity planning

The Simple Answer

Predictive scaling forecasts demand based on historical patterns. Instead of reactive autoscaling (scale up AFTER CPU hits 80%), predictive scaling provisions capacity BEFORE demand arrives. AWS Auto Scaling Predictive Scaling analyzes 14 days of historical data, identifies patterns (traffic peak at 9 AM, dip at midnight, surge on Mondays), and pre-scales to meet predicted demand. The result: no latency spike during traffic ramp-up because capacity is already provisioned.

Q31 Building an internal developer platform

The Simple Answer

An Internal Developer Platform (IDP) is a self-service layer that abstracts infrastructure complexity. Developers interact with the platform (through Backstage, CLI tools, or APIs) instead of directly managing Kubernetes, Terraform, and CI/CD. The platform team builds and maintains the abstractions.

Components

Service catalog (Backstage) - What exists, who owns it. Golden path templates (Backstage Templates) - How to create new things correctly. Self-service infrastructure (Backstage + Terraform/Crossplane) - Request a database, get one in 5 minutes without filing a ticket. CI/CD (standardized pipelines) - Build and deploy through consistent automation. Observability (Grafana/Datadog dashboards linked in Backstage) - See how things are running. Documentation (TechDocs) - Know how things work.

The Goal

Reduce cognitive load on developers. They should think about their business logic, not about how to configure Kubernetes RBAC, create a monitoring dashboard, or set up a CI/CD pipeline. The platform handles all of that.

Q32 DevEx - Developer Experience

The Simple Answer

DevEx measures how productive and happy developers are with their tools and processes. Key metrics: Time to first commit - How long until a new engineer makes their first production commit? (Target: <1 week). PR cycle time - From PR creation to merge. (Target: <24 hours). Deployment frequency - How often does the team deploy? (Target: multiple times per day). Build time - PR pipeline duration. (Target: <10 minutes). Onboarding time - Time to full productivity. (Target: <2 weeks).

Improving DevEx

Fast CI/CD (no one waits more than 10 minutes). Self-service infrastructure (no tickets for databases). Good documentation (answers findable in <5 minutes). Golden path templates (new service in 10 minutes). Reliable environments (staging that mirrors production).

Q33 AI governance in DevOps

The Simple Answer

As AI becomes embedded in DevOps tools, governance ensures: Security - AI-generated code may contain vulnerabilities. Review all AI suggestions. Do not paste AI outputs into production without security scanning. Prompt injection attacks can manipulate AI tools. Intellectual property - AI trained on open-source code may reproduce copyrighted patterns. Review for license compliance. Data privacy - Copilot and similar tools may send code to external APIs. Ensure sensitive code (credentials, proprietary algorithms) is not sent to AI services. Use self-hosted models or enterprise versions with data residency guarantees. Model drift - AI recommendations change over time as models are updated. Today's good suggestion may become tomorrow's anti-pattern. Maintain human oversight.

Q34 Collaboration at scale - platform teams

The Simple Answer

At 200+ engineers, collaboration patterns change. Stream-aligned teams own services end-to-end. Platform teams provide: shared infrastructure, CI/CD pipelines, monitoring, and developer tools. Enabling teams help stream-aligned teams adopt new technologies. Complicated subsystem teams manage complex domains (ML platform, data pipeline).

Inner Source

Apply open-source practices internally. Any engineer can contribute to any repository. Pull requests across teams. Shared ownership of common tools. Backstage provides discoverability - engineers find and contribute to projects across the organization.

SECTION 4 - Tricky Scenarios (Q35–Q40)

Q35 Copilot generated insecure code

What Happened

A developer used GitHub Copilot to generate a Terraform security group rule. Copilot suggested: `ingress { from_port = 22, to_port = 22, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] }`. This opens SSH to the entire internet. The code was committed, reviewed (reviewer missed it), and deployed. The server was compromised within hours.

Step-by-Step

Step 1 - Remediate immediately. Remove the 0.0.0.0/0 rule. Restrict to the company VPN CIDR. Check for signs of compromise on the instance.

Step 2 - Root cause. Copilot generated a common pattern - 0.0.0.0/0 appears frequently in tutorials and examples. The developer trusted the suggestion. The reviewer did not catch it.

Step 3 - Prevent. tfsec or Checkov in the CI pipeline catches overly permissive security groups before deployment. OPA policies reject 0.0.0.0/0 on port 22. AWS Config rules detect and auto-remediate. Train developers: AI suggestions are starting points, not finished code. Security scanning is the mandatory safety net.

Q36 ChatOps bot deployed without approval

What Happened

A developer configured the Slack bot to deploy on the `/deploy` command. There was no approval gate. A developer typed `/deploy api v2.1.0 production` accidentally (meant to type staging). The untested version went to production. Outage.

Step-by-Step

Step 1 - Rollback to the previous production version.

Step 2 - Fix the bot. Add mandatory confirmation: "You are deploying api v2.1.0 to PRODUCTION. Type 'CONFIRM' to proceed." Add approval workflow: production deploys require a second person to approve (Slack button). Restrict production deploy permission to the release-managers Slack user group.

Step 3 - Add guardrails. The bot should verify: the version exists in the staging registry (was it tested?), the image passed security scans, and the deployment has been approved in Jira.

Q37 AI monitoring alert fatigue

What Happened

The team enabled AI anomaly detection on all metrics. The ML model flags 50+ "anomalies" per day. Most are normal variations - a traffic dip during lunch, a CPU spike during cron jobs, a memory increase during deployments. Engineers start ignoring all AI alerts. A real anomaly (memory leak) is missed because it is buried in noise.

Step-by-Step

Step 1 - Tune the sensitivity. Increase the anomaly detection threshold - only alert on deviations greater than 3 standard deviations (instead of 2). Reduce false positives at the cost of slightly slower detection.

Step 2 - Add context. Suppress anomalies during known events: deployments, cron jobs, planned maintenance windows. Mark these periods so the ML model excludes them.

Step 3 - Progressive alerting. Anomaly detected → log it (Level 1). Anomaly persists for 5 minutes → Slack notification (Level 2). Anomaly persists for 15 minutes → page on-call (Level 3). Transient anomalies self-resolve without paging anyone.

Step 4 - Feedback loop. When engineers dismiss an alert, they label it: "false positive - deployment," "false positive - expected traffic." Feed these labels back to the ML model. Over time, the model learns which patterns are normal and stops alerting on them.

Q38 Backstage adoption failed

What Happened

The platform team built a Backstage portal. Six months later, only 10% of developers use it. Most still search Slack, GitHub, and Confluence directly.

Step-by-Step

Step 1 - Understand why. Survey developers. Common reasons: "I did not know it existed." "The catalog is incomplete - my service is not there." "The data is stale." "It does not have the information I need."

Step 2 - Solve discoverability. Make Backstage the homepage for engineering browsers. Link to Backstage from Slack bot responses: "You asked about the payment service → here's the Backstage page."

Step 3 - Ensure completeness. Require every service to register in the catalog. Add catalog registration to the CI pipeline - if catalog-info.yaml is missing, the pipeline warns. Import existing services from GitHub/GitLab automatically.

Step 4 - Make it the source of truth. Integrations must be live - not stale snapshots. Kubernetes plugin shows real-time Pod status. GitHub Actions plugin shows the latest pipeline run. If the data is fresh and accurate, developers will trust and use it.

Step 5 - Add value beyond a directory. Software templates (golden paths) give developers a reason to START in Backstage. "I need a new service" → Backstage creates everything in 5 minutes. This creates the habit of going to Backstage first.

Q39 Postmortem action items never completed

What Happened

The team conducts thorough postmortems after every incident. The reports are well-written with clear action items. But 3 months later, only 20% of action items are completed. The same incidents repeat because the underlying issues are never fixed.

Step-by-Step

Step 1 - Make action items Jira tickets. Not bullet points in a Confluence document - actual Jira tickets with: assignee, due date, priority, and the postmortem link. Track in the team's sprint backlog alongside feature work.

Step 2 - Allocate capacity. Dedicate 20% of sprint capacity to reliability work (postmortem action items, tech debt, monitoring improvements). If all capacity goes to features, action items never get done.

Step 3 - Weekly review. In the team standup or weekly meeting, review open postmortem action items. "We have 5 open items from the last 3 incidents. JIRA-456 is overdue - what's blocking it?"

Step 4 - Escalation. If an action item is overdue by 2 weeks, escalate to the engineering manager. If the same incident type recurs because the action item was not completed, this is a P1 priority - the cost of NOT fixing it is now measured in repeated outages.

Step 5 - Track metrics. Postmortem action item completion rate. Time to completion. Repeat incident rate. Share these metrics with leadership - they demonstrate the value (or failure) of the postmortem process.

Q40 Design a developer experience platform for 200 engineers

The Complete Architecture

Portal: Backstage - Software catalog (all 150 services), TechDocs (documentation from repos), software templates (golden paths for Python, Java, Go services), plugins for: Kubernetes, GitHub Actions, PagerDuty, Grafana, SonarQube, Kubecost.

Golden path templates - "Create Python microservice," "Create React frontend," "Create data pipeline." Each generates: GitHub repo, Dockerfile, Helm chart, CI/CD pipeline, Prometheus ServiceMonitor, Grafana dashboard, PagerDuty service, and Backstage catalog entry.

CI/CD: GitHub Actions + ArgoCD - Standardized workflow templates in a shared .github repository. ArgoCD for GitOps deployment. Developers do not write CI/CD from scratch - they use the template.

Infrastructure self-service - Backstage template: "Request a PostgreSQL database" → creates Terraform resource, applies via CI, updates Backstage catalog. No ticket to the platform team.

Observability - Grafana dashboards auto-created per service. Prometheus metrics standardized via library. PagerDuty on-call visible in Backstage.

Collaboration - Slack with ChatOps bot for deployments and incidents. Jira with smart commits and deployment tracking. Confluence for postmortems and ADRs. All linked in Backstage.

Metrics to track - Time to first commit: under 3 days. PR cycle time: under 12 hours. Build time: under 8 minutes. New service creation: under 15 minutes. Onboarding: under 1 week to first production deploy. Developer satisfaction: quarterly survey, target NPS > 40.

Interview Tips

For Junior Roles

Focus on Q1-Q10. Know what Copilot does and its limitations. Understand ChatOps and Jira integration with CI/CD. Know what Backstage is at a high level. Explain why blameless postmortems matter.

For Mid-Level Roles

Q1-Q28. Be ready to discuss: AI for monitoring (anomaly detection), Backstage templates (golden paths), PagerDuty escalation policies, and runbook automation levels. Know ChatOps deployment approval patterns.

For Senior Roles

All 40 questions. Internal developer platform design, DevEx metrics, AI governance, and collaboration at scale separate senior from mid-level. Be ready to design a complete developer experience for a large organization (Q40).

Quick Reference Cheat Sheet

Topic	Key Takeaway
AIOps	AI augments engineers - anomaly detection, noise reduction, root cause suggestion.
GitHub Copilot	AI pair programmer. Great for IaC boilerplate. ALWAYS review for security.
CodeWhisperer/Q	AWS-specialized AI. Better than Copilot for Boto3 and CloudFormation.
AI code review	First pass for mechanical issues. Human review for architecture and logic.
ChatOps	Operations in Slack. Visible, documented, collaborative. Add approval gates.
Jira	Link tickets to commits, builds, and deployments. Smart commits. Automation rules.
Confluence	Runbooks, postmortems, ADRs. Documentation close to code in Git, broader in Confluence.
Backstage	Developer portal. Software catalog + templates + TechDocs + plugins. CNCF.
Golden paths	Standardized templates for new services. Consistency. Compliance built in.
PagerDuty	On-call schedules, escalation policies, incident workflows. The standard.
Statuspage	Public incident communication. Reduces support tickets 80% during outages.
Postmortems	Blameless. Focus on systems not people. Action items in Jira. Track completion.
Runbook automation	Wiki → semi-automated script → fully automated. Progress incrementally.
AI anomaly detection	Learns normal patterns. Alerts on deviations. Tune sensitivity. Feedback loop.
AI cost optimization	Spot instances (Spot.io), right-sizing (Kubecost), predictive scaling.
Developer platform	Backstage + templates + self-service infra + standard CI/CD + observability.
DevEx metrics	Time to first commit, PR cycle time, build time, deploy frequency, satisfaction.
AI governance	Review AI output. Security scan. Data privacy. IP compliance. Human oversight.